

The Brain

A second-brain Obsidian vault, wired into Claude Code

Hugo's working memory across five active projects: Gaussion Portal, Neware Agent, Magnetics, Design Support, Business Development, plus the personal Slubstack build. Every project's live state, decisions, problems, ideas, and per-session logs live in one git-backed Obsidian vault at `~/Brain`. Claude Code reads it at session start through a custom hook, so context loads automatically and never has to be re-explained.

Author: Hugo Idle · Vault repo: [Hugoldle/Brain](#) · Generated 2026-05-04

1. What The Brain is

The Brain is a personal knowledge system kept as a folder of Markdown files in `~/Brain`, viewed in Obsidian and version-controlled in a private GitHub repository. It is the single source of truth for everything that does not belong inside a code repository: project state, decisions, recurring problems, half-formed ideas, daily journal entries, and per-session logs.

Every Markdown file is connected to others via Obsidian's `[[wikilinks]]`, which means the graph view stays navigable and unrelated files do not become orphan nodes. Conventions are enforced lightly: the project hub file is named after the project (so it shows up clearly in the graph), `progress.md` is the live source of truth (with `roadmap.md` as the aspirational counterpart), and the canonical pattern for state files repeats across all projects.

The Brain is also a self-improving system. The `/distill` command mines the last fortnight of session logs for recurring corrections and patterns, then proposes updates to `me.md`, the relevant `working-style.md`, and Claude Code's auto-memory directory. Updates happen with explicit approval, so the system learns deliberately rather than drifting.

Why it exists

- **Continuity across sessions.** Claude Code conversations are ephemeral. The Brain is the durable substrate.
- **Continuity across projects.** Five projects in flight at once. Without a fixed schema, context-switching loses an hour every time.
- **Pattern visibility.** Recurring problems, repeated decisions, and stalling ideas only become visible once they are written somewhere consistent.
- **Replayable state.** Any session, on any machine, can be re-entered with the right context loaded automatically.

2. File hierarchy

The vault structure is intentionally shallow and predictable. Files have fixed names across projects so tooling, hooks, and slash commands can find them without configuration.

Top level

```
~/Brain/
  index.md entry point, links to all projects
  me.md user profile, working style, conventions
  Daily/
    YYYY-MM-DD.md journal, sessions list, notes
  Inbox/
    YYYY-MM-DDTHH-MM-SS.md mobile captures awaiting filing
  Projects/
    work/gaussian/ five-project umbrella
    personal/ slubstack, sicily-2026, etc.
    kernel/ Brain meta-project
  .meta/
    last-distill.txt cadence stamp for /distill
```

Per-project skeleton

Every project folder repeats the same pattern. The hub file is named after the project itself (not `overview.md`) so the project node is visible in Obsidian's graph view.

```
Projects/work/gaussian/gaussian-portal/
  gaussian-portal.md hub, links everything else
  progress.md source of truth, live state, current focus
  roadmap.md aspirational, updated FROM progress at wrap
  decisions.md logged mid-session as they happen
  problems.md Outstanding / Resolved counts maintained
  ideas.md half-formed thoughts, dated
  working-style.md project conventions and gotchas
  sessions/
    YYYY-MM-DD.md per-session log written by /wrap
```

File responsibilities

progress.md	Live state. The single source of truth. When progress and roadmap disagree, trust progress.
roadmap.md	Aspirational. Where the project is heading. Synced from progress at wrap-up only.
decisions.md	Append-only log of product or technical decisions. Logged the moment they happen, not at wrap.
problems.md	Bugs, UX issues, things broken. Outstanding/Resolved tally kept current at the top.
ideas.md	Half-formed thoughts not yet committed to roadmap. Dated. Stale ideas surface in /distill.
working-style.md	Project-specific conventions, recurring gotchas, anything more local than me.md.
sessions/	One file per session. Written by /wrap. Contains goal, what was shipped, what is next.

3. Slash commands

Six Brain-related commands cover the full workflow. All are invoked with `/<name>` inside Claude Code. None of them write speculatively; all of them ask before destructive operations.

Mid-session capture

/log	Quick-capture an idea, problem, or decision into the current project's vault file. No ceremony. Use when something surfaces during work that should not wait for <code>/wrap</code> .
/file-inbox	Sort mobile captures from <code>~/Brain/Inbox/</code> into their proper homes (problems, decisions, ideas, working-style, daily). Runs first thing in any session that has unfiled captures, with a confirmation step before writing.
/ship	Stage modified or untracked files relevant to the app and create a clean commit, excluding test artefacts (playwright-report, test-results, screenshots). Used when the change is ready for the deploy pipeline.

Session close and weekly cadence

/wrap	Lean session close. Writes the session file, updates <code>progress.md</code> current-focus, recounts outstanding/resolved problems, and stamps the daily note. No pattern mining; that is <code>/distill</code> 's job. Includes a one-line nudge if <code>/distill</code> is more than 7 days overdue.
/distill	Weekly self-improvement pass. Mines the last 14 days of session logs for recurring corrections, gotchas, stale entries, problem clusters, and orphan ideas. Proposes updates to <code>me.md</code> , the relevant <code>working-style.md</code> , and auto-memory for per-item approval. Stamps <code>last-distill.txt</code> on completion.
/briefing	Weekly Monday line-manager export. Synthesises status across the four Gausson work projects from <code>progress.md</code> , recent <code>Daily/</code> notes, and outstanding <code>problems.md</code> entries. Produces the kind of update a line manager wants to read on a Monday morning.

How they relate

/log and **/file-inbox** feed the system with raw input. **/wrap** reconciles a single session against the system. **/ship** connects the Brain to deployment. **/distill** is the weekly self-loop: the Brain teaches itself. **/briefing** is the weekly outward loop: the Brain teaches Hugo's manager.

4. Session-start auto-injection

When Claude Code starts a session, a hook script runs at `~/claude/scripts/session-context.sh`. Its job is to load relevant Brain context into the conversation automatically, so Hugo never has to re-explain what was happening yesterday.

What fires, in order

Brain pull	<code>git pull --ff-only</code> with a 5-second timeout, silent on success. Keeps the vault in sync across machines.
Inbox check	Counts unfiled mobile captures in <code>Inbox/</code> . If any exist, emits a strong nudge so Claude runs <code>/file-inbox</code> first.
Stale ideas	Counts idea bullets older than 7 days across all projects. Reports a soft nudge if any.
Project context	Resolved from <code>cwd</code> . Loads current focus from <code>progress.md</code> , top three traps from <code>working-style.md</code> , outstanding/resolved problem counts, and a link to the latest session file.
Resume detection	If the latest session file has a <i>Next pickup</i> block (written by <code>/wrap</code>) or contains phrases like 'paused before' or 'resume from', extract that and surface it as resume context.
Yesterday's daily line	Pulls the bullet for this project from yesterday's daily note, for cross-project continuity.
Distill due	If <code>last-distill.txt</code> is more than 7 days old, nudge to run <code>/distill</code> .
Intent prompt	Only fires when no resume context was detected. Prompts Hugo for the session goal, used as the title in the next <code>/wrap</code> session log.

What this replaces

Without this hook, every session would begin with a 'where were we?' prompt and five minutes of Hugo re-explaining yesterday's state. The hook turns that into a single short greeting from Claude that already references the live focus and any outstanding traps. The cost is a few hundred tokens of injection at startup, amortised over a typical session.

5. Future possibilities

The Brain today is well-developed but mostly reactive: it captures what has happened and surfaces context on demand. The next phase is for it to become predictive and proactive. Here is what becomes possible if the system keeps maturing.

Near-term, already planned

- **iOS Shortcuts to GitHub API.** Append voice notes to `Daily/YYYY-MM-DD.md` from the phone, no Claude API or new apps required. Uses a fine-grained PAT scoped to the Brain repo only.
- **Better mobile transcription.** The current mobile-capture flow produces transcripts with garbled names (a real example today read 'florist Chang' instead of 'Forrest Chang'). Either an on-device model or a post-processing step that corrects against known nouns in the vault would cut filing-friction noticeably.
- **Multi-modal capture.** Photos of whiteboards, screenshots of designs, sketches of battery rigs — all into Inbox, all routed by `/file-inbox` to the right project.

Medium-term, plausible

- **Predictive pattern matching.** Before a feature merges, surface similar past features and any problems they caused. *'You are about to ship a feature similar to one that hit problem #61 last month — review that pattern first.'*
- **Decision-outcome correlation.** Decisions are logged with date and rationale. If a project hits a regression, the system can show which prior decisions are most likely upstream. Over time this becomes a structured account of what reasoning actually pays off.
- **Cross-project knowledge graph queries.** 'Show me every problem across all projects that touches Tailwind v4.' 'Find every decision involving Cosmos DB indexes.' The vault is already a graph; making it queryable beyond Obsidian's native search unlocks compound insight.
- **Auto-generated weekly briefings.** `/briefing` today is template-driven. With richer state, it can write the narrative — what was the week's main risk, what unblocked, what will probably slip — in language the manager already reads.
- **Time-series project views.** 'How did Portal velocity change after Phase B?' 'How long has problem cluster X been open?' Answers come from session-log timestamps without any new instrumentation.

Longer-term, ambitious

- **Self-pruning auto-memory.** `/distill` already proposes prunes for stale entries. Eventually the system can prune autonomously with a confidence threshold and an audit trail, leaving Hugo to approve only edge cases.
- **Skill auto-creation from repeated patterns.** When the same correction lands in `/distill` three weeks running, propose a new skill or hook that prevents the correction from being needed in the first place. The Brain would author its own working-style entries and even its own slash commands.
- **Calendar and task integration.** Pull tomorrow's meetings into today's daily note. Surface upcoming deadlines against open problems. Connect Linear/Asana state to the same graph the Brain already maintains.

- **Outcome scoring.** When a problem closes, log how long it was open and what fixed it. Over many problems, surface what classes of bug take longest. Direct future preventative investment based on data, not impression.
- **Multi-agent collaboration.** Today the Brain serves a single Claude Code session. Tomorrow it could be the shared substrate for multiple specialised agents (a security-review agent, a deploy-watcher agent, a meeting-scribe agent), each writing to and reading from the same vault, with conflict resolution baked in.

The honest constraint

All of the above is feasible because the substrate is already healthy: Markdown files, predictable schemas, git history, wikilinks for the graph. The constraint is not technical capability — it is restraint. Most second-brain systems collapse not from missing features but from feature inflation: too many file types, too many slash commands, too many automated nudges. The Brain's biggest competitive advantage today is that it stops short. `/wrap` does not try to also be `/distill`. `me.md` is not allowed to bloat with project-specific rules. The discipline of doing less is the thing that lets it scale at all.

Generated by Claude Code using the document-skills:pdf skill. Source: Hugo's CLAUDE.md, me.md, and `~/claude/scripts/session-context.sh` as of 2026-05-04.